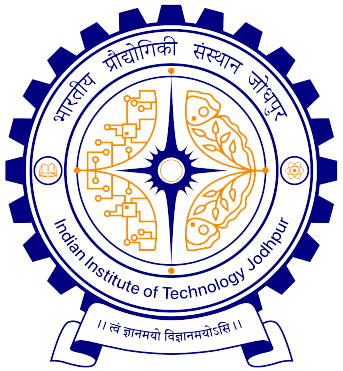
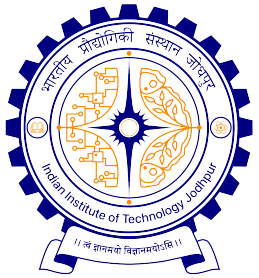


Introduction to Computer Science (CSL1010)

C Basics– Pointers L13

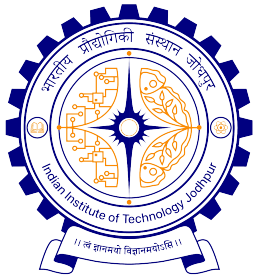
Suchetana Chakraborty
Department of Computer Science, IIT Jodhpur
Spring, 2026



Pointers – Basic Concept

- As seen before, in memory, every stored data item occupies one or more contiguous memory cells
  - The number of memory cells required to store a data item depends on its type (char, int, double, etc.).

- Whenever we declare a variable, the system allocates memory location(s) to hold the value of the variable.
  - Since every byte in memory has a unique address, this location will also have its own (unique) address.



What is a pointer?

- It is a variable, just like other variables you studied
  - So it has type, storage etc.
- Difference: it can only store the address (rather than the value) of a data item
- Type of a pointer variable = type of the data whose address it will store
  - Example: int pointer, float pointer,…
  - Can be pointer to any user-defined types also like structure types

What is a pointer?

A pointer is a variable that represents the location (rather than the value) of a data item.

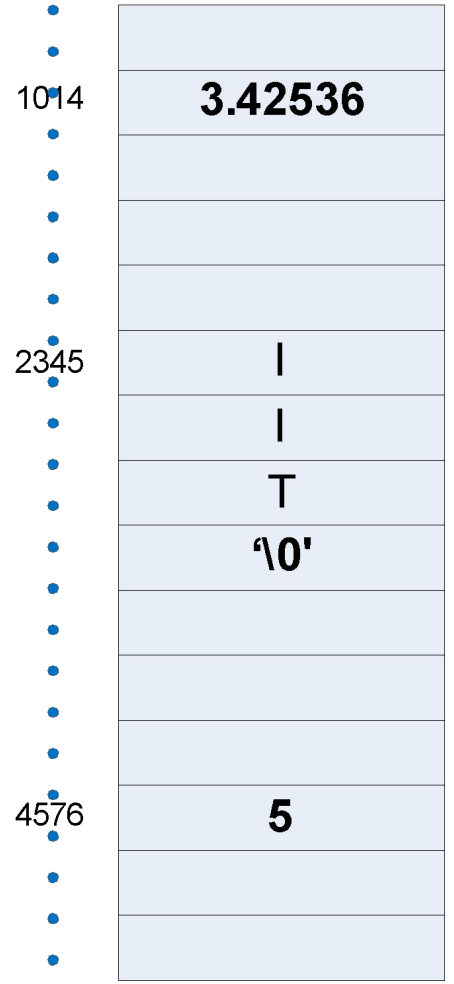
```
int y = 5;
char name[5]="IIT";
float x;
```

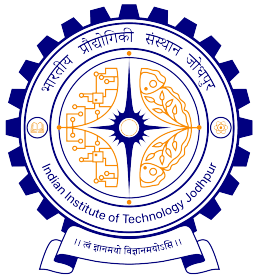
Memory locations

&y = 4576

&x = 1014

&name[0] = 2345





Application of Pointers

- They have a number of useful applications:
  - Enables us to access a variable that is defined outside the function
  - Can be used to pass information back and forth between a function and its reference point
  - More efficient in handling data tables
  - Reduces the length and complexity of a program
  - Sometimes also increases the execution speed

Pointer Variable

- Consider the statement

```
int  xyz = 50;
```

  - This statement instructs the compiler to allocate a location for the integer variable xyz, and put the value 50 in that location

  - Suppose that the address location chosen is 1380

xyz	□	**variable**
50	□	**value**
1380	□	**address**

Pointer Variable

- During execution of the program, the system always associates the name xyz with the address 1380
  - The value 50 can be accessed by using either the name xyz or the address 1380

- Since memory addresses are simply numbers, they can be assigned to some variables which can be stored in memory
  - Such variables that hold memory addresses are called pointers
  - Since a pointer is a variable, its value is also stored in some memory location

Pointer Variable

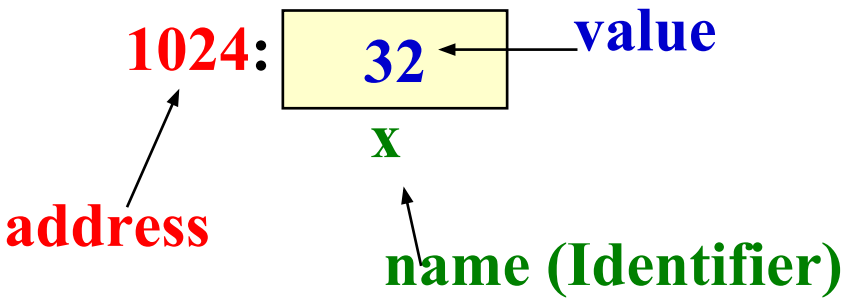
- Suppose we assign the address of xyz to a variable p
  - p is said to point to the variable xyz

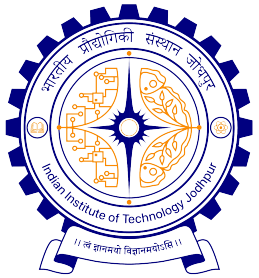
Variable	Value	Address
xyz	50	1380
p	1380	2545

p = &xyz;

Values vs Locations

- Variables name memory locations, which hold values



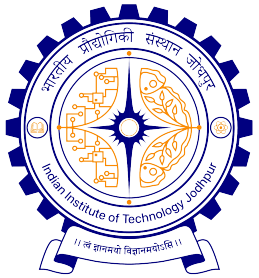


Pointers

- A pointer is just a C variable whose value can contain the address of another variable
- Needs to be declared before use just like any other variable
- General form:

```
data_type *pointer_name;
```

- Three things are specified in the above declaration:
  - The asterisk (*) tells that the variable pointer_name is a pointer variable
  - pointer_name needs a memory location
  - pointer_name points to a variable of type data_type



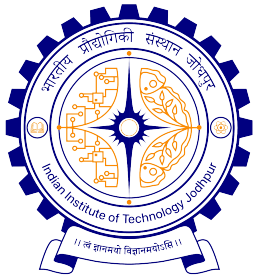
Example

```
int *count;
float *speed;
char *c;
```

- Once a pointer variable has been declared, it can be made to point to a variable using an assignment statement like

```
int *p, xyz;
    :
p = &xyz;
```

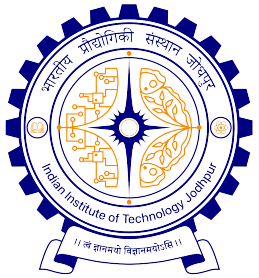
- This is called pointer initialization



- Pointers can be defined for any type, including user defined types
- Example

```
struct name {
    char first[20];
    char last[20];
};
struct name *p;
```

- p is a pointer which can store the address of a struct name type variable



Accessing the Address of a Variable

- The address of a variable is given by the & operator
  - The operator & immediately preceding a variable returns the address of the variable

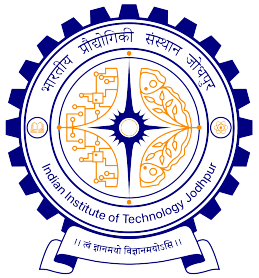
- Example:

```
p = &xyz;
```

  - The address of xyz (1380) is assigned to p

- The & operator can be used only with a simple variable (of any type, including user-defined types) or an array element

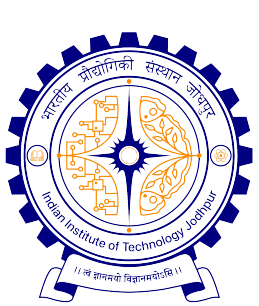
```
&distance
&x[0]
&x[i-2]
```



Illegal Use of &

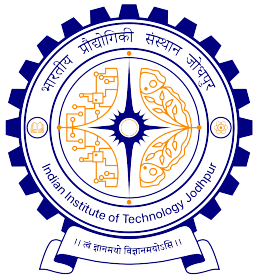
- &235
  - Pointing at constant
- int arr[20];
  :
  &arr;
  - Pointing at array name
- &(a+b)
  - Pointing at expression

In all these cases, there is no storage, so no address either



Example

```
#include  <stdio.h>
int main(){
    int   a;
    float  b, c;
    double  d;
    char  ch;

    a = 10;   b = 2.5;  c = 12.36;  d = 12345.66;  ch = 'A';
    printf  ("%d is stored in location %u \n",  a,  &a) ;
    printf  ("%f is stored in location %u \n",  b,  &b) ;
    printf  ("%f is stored in location %u \n",  c,  &c) ;
    printf  ("%lf is stored in location %u \n", d,  &d) ;
    printf  ("%c is stored in location %u \n",  ch, &ch) ;
    return 0;
}
```



Output

```
10 is stored in location 3221224908
2.500000 is stored in location 3221224904
12.360000 is stored in location 3221224900
12345.660000 is stored in location 3221224892
A is stored in location 3221224891
```

Accessing a Variable Through its Pointer

- Once a pointer has been assigned the address of a variable, the value of the variable can be accessed using the indirection operator (*).

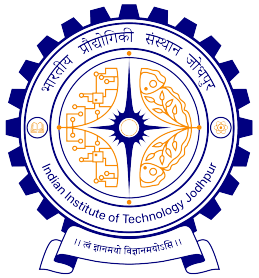
```
int   a, b;
int   *p;
p = &a;
b = *p;
```

Equivalent to

```
b = a;
```

Note:

b = *(&a); is a short-cut!

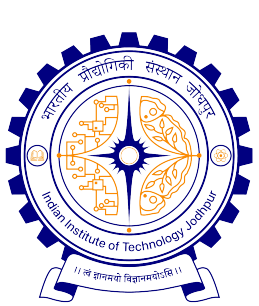


Example

```
#include  <stdio.h>
int main()
{
    int   a, b;
    int   c = 5;
    int   *p;

    a  =  4  *  (c  +  5) ;

    p  =  &c;
    b  =  4  *  (*p  +  5) ;
    printf  ("a=%d  b=%d \n",  a, b);
    return 0;
}
```

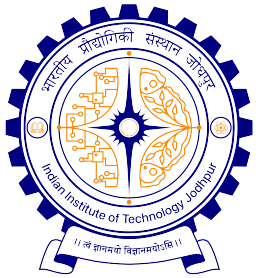
Equivalent

```
a=40 b=40
```



Example

```
int main()
{
    int  x, y;
    int  *ptr;

    x = 10 ;
    ptr = &x ;
    y = *ptr ;
    printf ("%d is stored in location %u \n",  x,  &x);
    printf ("%d is stored in location %u \n",  *&x,  &x);
    printf ("%d is stored in location %u \n",  *ptr,  ptr);
    printf ("%d is stored in location %u \n",  y,  &*ptr);
    printf ("%u is stored in location %u \n",  ptr, &ptr);
    printf ("%d is stored in location %u \n",  y,  &y);

    *ptr = 25;
    printf ("\nNow x = %d \n", x);
    return 0;
}
```



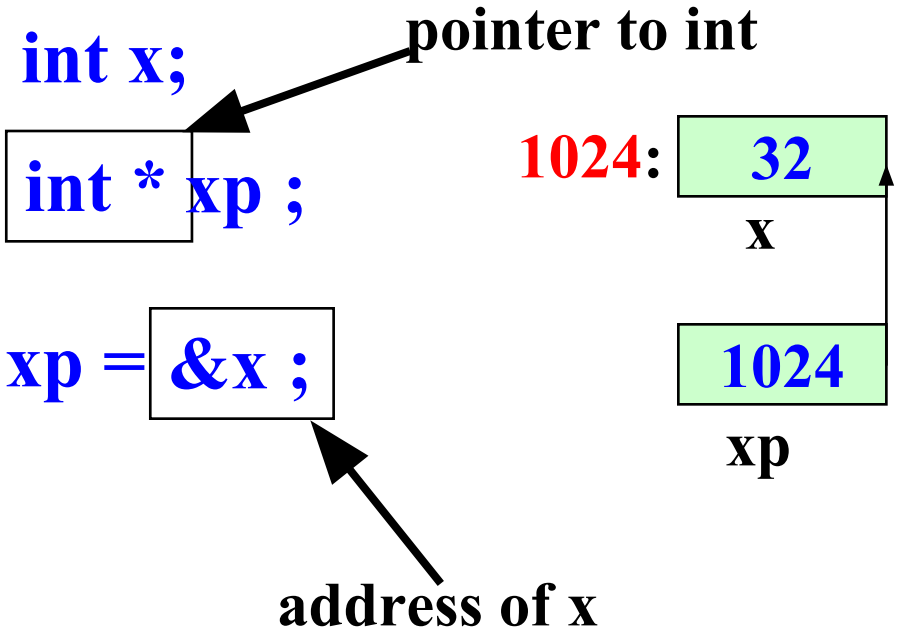
Suppose that

```
Address of x:      1833513640
Address of y:      1833513636
Address of ptr:    1833513624
```

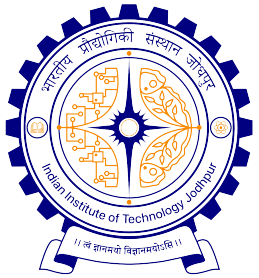
Then output is

```
10 is stored in location 1833513640
10 is stored in location 1833513640
10 is stored in location 1833513640
10 is stored in location 1833513640
1833513640 is stored in location 1833513624
10 is stored in location 1833513636


Now x = 25
```

Example




```
*xp = 0;        /* Assign 0 to x */
*xp = *xp + 1;  /* Add 1 to x */
```



Value of the pointer

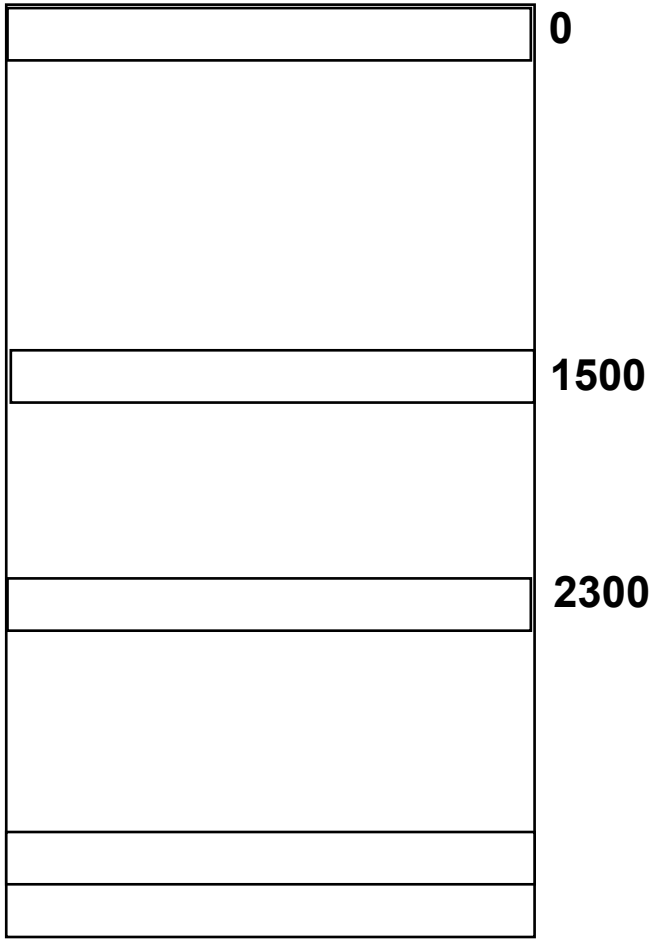
- Declaring a pointer just allocates space to hold the pointer – it does not allocate something to be pointed to!
  - Local variables in C are not initialized, they may contain anything

- After declaring a pointer:

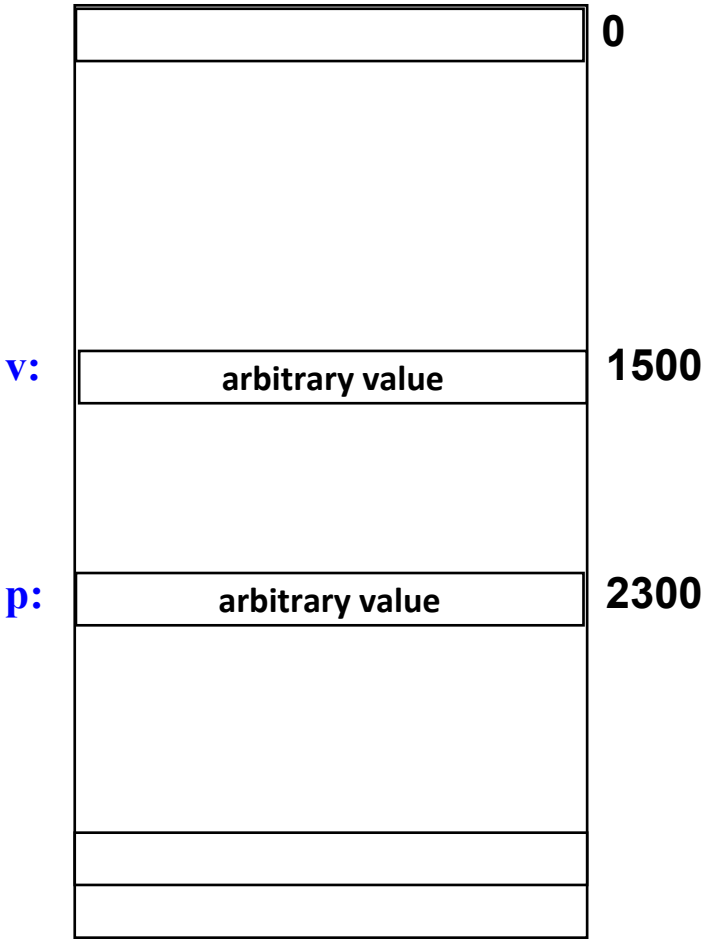
```
int *ptr;
```

ptr doesn't actually point to anything yet. We can either:
  - make it point to something that already exists, or
  - allocate room in memory for something new that it will point to… (dynamic allocation, to be done later)

Example

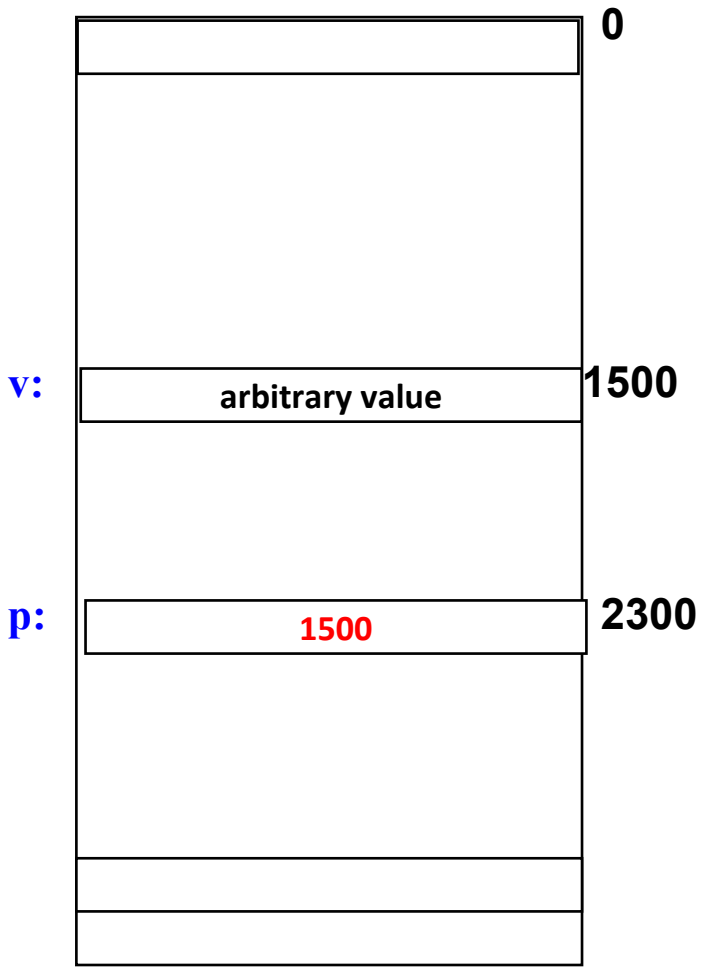


Memory and Pointers:



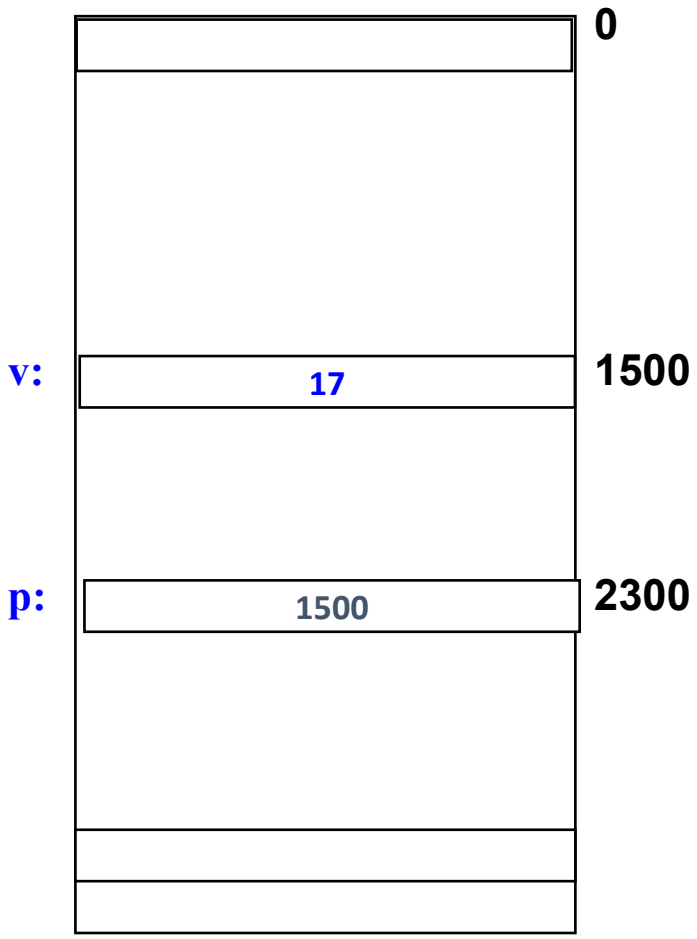
Memory and Pointers:

```
int *p, v;
```



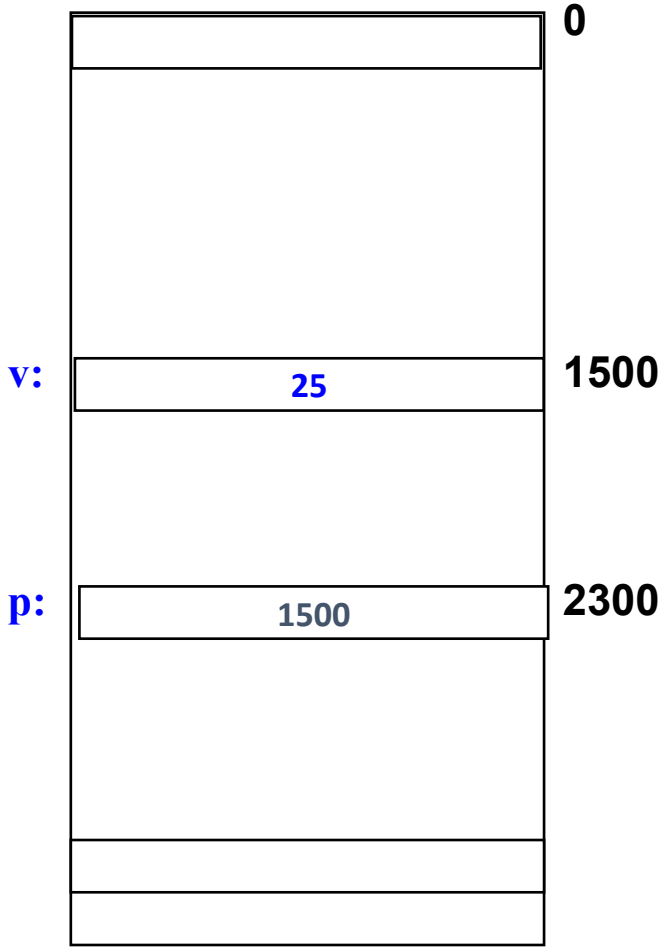
Memory and Pointers:

```
int v, *p;
p = &v;
```



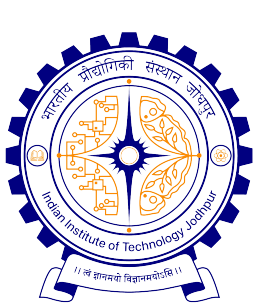
Memory and Pointers:

```
int v, *p;
p = &v;
v = 17;
```



Memory and Pointers:

```
int v, *p;
p = &v;
v = 17;
*p = *p + 4;
v = *p + 4
```



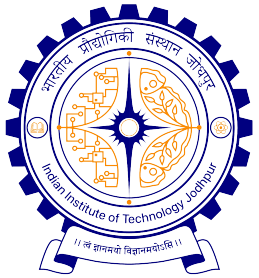
More Examples: Pointers in Expressions

If p1 and p2 are two pointers, the following statements are valid:

```
sum = *p1 + *p2;

prod = *p1 * *p2;

prod = (*p1) * (*p2);

*p1 = *p1 + 2;

x = *p1 / *p2 + 5;
```

`*p1` can appear as l-value

- Note that this unary * has higher precedence than all arithmetic / relational / logical operators



Things to Remember

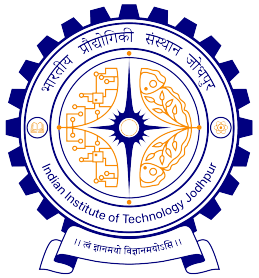
- Pointer variables must always point to a data item of the same type

```
float   x;
int    *p;
    :
p = &x;
```

will result in wrong output

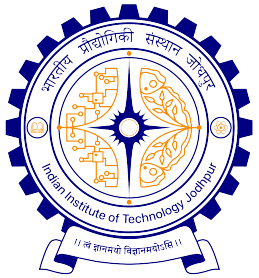
- Never assign an absolute address to a pointer variable

```
int   *count;
count = 1268;
```



Pointer Expressions

- Like other variables, pointer variables can appear in expressions
- What are allowed in C?
  - Add an integer to a pointer
  - Subtract an integer from a pointer
  - Subtract one pointer from another (related)
    - If p1 and p2 are both pointers to the same array, then p2 – p1 gives the number of elements between p1 and p2



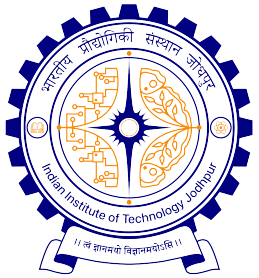
Contd.

- What are NOT allowed?
  - Adding two pointers.

    p1 = p1 + p2;
  - Multiply / divide a pointer in an expression

    p1 = p2 / 5;

    p1 = p1 – p2 * 10;

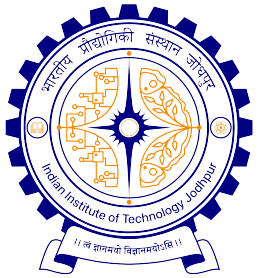


Pointer Arithmetic

Like other variables, pointer variables can be used in arithmetic expressions. Suppose, p1 is a pointer to x and p2 is a pointer to y.

Examples:

```
*p1 = *p2 + 10;          // Same as x = y + 10;
 y = *p2 + 1;            // Same as y = ++y;
*p1 += 1;                // Same as x = x + 1;
++(*p1);                 // Same as *p1 = *p1+1;
*p1 = *p2;               // Same as x = y;
 p1 = p2;                // p1 and p2 both points to y;
```

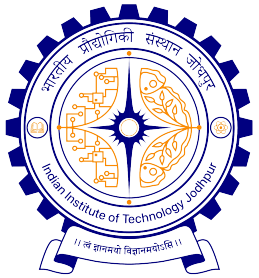


Scale Factor

We have seen that an integer value can be added to or subtracted from a pointer variable

```
int  *p1, *p2;
int  i, j;
     :
p1 = p1 + 1;
p2 = p1 + j;
p2++;
p2 = p2 – (i + j);
```

- In reality, it is not the integer value which is added/subtracted, but rather the scale factor times the value



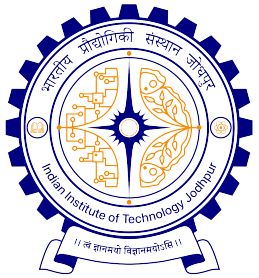
Contd.

Data Type	Scale Factor
char	1
int	4
float	4
double	8

- If p1 is an integer pointer, then

  p1++

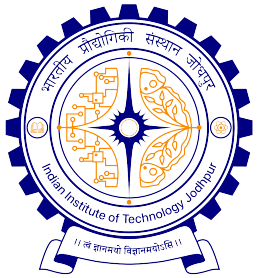
  will increment the value of p1 by 4



Contd.

- The scale factor indicates the number of bytes used to store a value of that type
  - So the address of the next element of that type can only be at the (**current pointer value + size of data**)
- The exact scale factor may vary from one machine to another
- Can be found out using the sizeof function
  - Gives the size of that data type
- Syntax:

  sizeof (data_type)

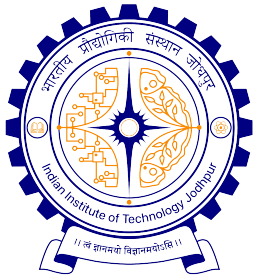


Example

```
int main(){
  printf ("No. of bytes in int is %u \n",         sizeof(int));
  printf ("No. of bytes in float is %u \n",      sizeof(float));
  printf ("No. of bytes in double is %u \n", sizeof(double));
  printf ("No. of bytes in char is %u \n",      sizeof(char));

  printf ("No. of bytes in int * is %u \n",          sizeof(int *));
  printf ("No. of bytes in float * is %u \n",       sizeof(float *));
  printf ("No. of bytes in double * is %u \n",  sizeof(double *));
  printf ("No. of bytes in char * is %u \n",       sizeof(char *));
  return 0;
}
```

Output on a PC

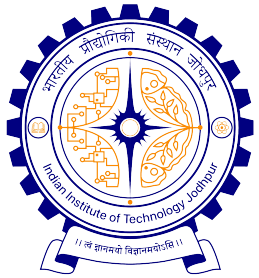
```
No. of bytes in int is 4
No. of bytes in float is 4
No. of bytes in double is 8
No. of bytes in char is 1
No. of bytes in int * is 4
No. of bytes in float * is 4
No. of bytes in double * is 4
No. of bytes in char * is 4
```



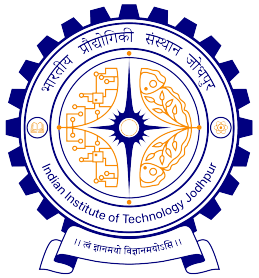
- Note that pointer takes 4 bytes to store, independent of the type it points to
- However, this can vary between machines
  - Output of the same program on a server

```
No. of bytes in int is 4
No. of bytes in float is 4
No. of bytes in double is 8
No. of bytes in char is 1
No. of bytes in int * is 8
No. of bytes in float * is 8
No. of bytes in double * is 8
No. of bytes in char * is 8
```

- Always use sizeof() to get the correct size`
- Should also print pointers using **%p** (instead of %u as we have used so far for easy comparison)

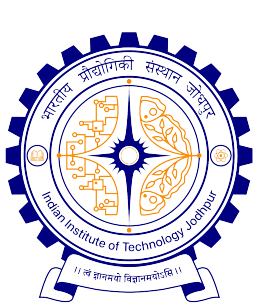


Pointers and Arrays



Pointers and Arrays

- When an array is declared,
  - The compiler allocates sufficient amount of storage to contain all the elements of the array in contiguous memory locations
  - The base address is the location of the first element (index 0) of the array
  - The compiler also defines the array name as a constant pointer to the first element
- In C, there is a strong relationship between pointers and arrays.
  - Any operation that can be achieved by array subscripting can also be done with pointers
  - The pointer version, in general, is faster.



Example

- Consider the declaration:

  int x[5] = {1, 2, 3, 4, 5};

- Suppose that each integer requires 4 bytes
- Compiler allocates a contiguous storage of size 5x4 = 20 bytes
- Suppose the starting address of that storage is 2500

Element	Value	Address
x[0]	1	2500
x[1]	2	2504
x[2]	3	2508
x[3]	4	2512
x[4]	5	2516

Array Name is a Constant Pointer

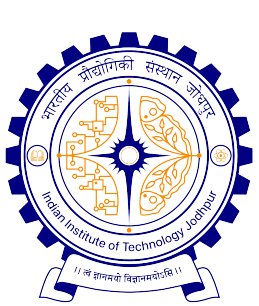
- The array name x is the starting address of the array
  - Both x and &x[0] have the value 2500
  - x is a constant pointer, so cannot be changed

    - *x = 3400, x++, x += 2 are all illegal*

- If int *p is declared, then p = x; and p = &x[0]; are equivalent

- We can access successive values of x by using p++ or p-- to move from one element to another

- p and x points to same memory location; however, x is not a pointer variable! (p+i) is the address of x[i].



Array Name x and Pointer p

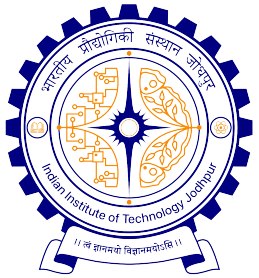
- Relationship between p and x:

```
p    =  &x[0]  =  2500
p+1  =  &x[1]  =  2504
p+2  =  &x[2]  =  2508
p+3  =  &x[3]  =  2512
p+4  =  &x[4]  =  2516
```

In general, *(p+i) gives the value of x[i]

- C knows the type of each element in array x, so knows how many bytes to move the pointer to get to the next element

- This is why pointers have types (like int pointers, char pointers). They are not just a single "address" data type.



Array Name x and Pointer p

- If p points to an array x, then
  p = x; implies the current value of the pointer
  *(p+i) implies the content in x[i]

- Few things to be noted

  x+i is also identical to p+i

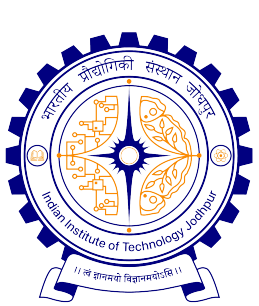
  x[i] can also be written as *(x+i)

  &x[i] and x+i are identical

  p[i] is identical to *(p+i)

  p = x and p++ are legal *// Because, p is a pointer variable*

  x = p and x++ are illegal *// name of an array is not a*



Example

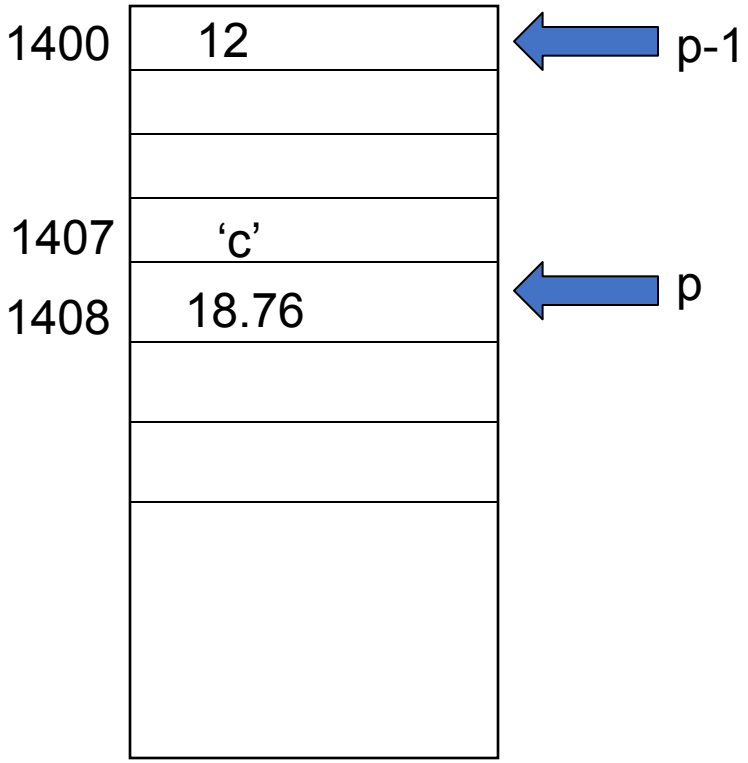
```
int main(){
  int A[5],  i;

  printf("The addresses of the array elements are:\n");
  for (i=0; i<5; i++)
     printf("&A[%d]: Using \%p = %p,   Using \%u = %u", i, &A[i], &A[i]);
  return 0;
}
```

Output on a server machine

&A[0]: Using %p = 0x7fffb2ad5930, Using %u = 2997705008
&A[1]: Using %p = 0x7fffb2ad5934, Using %u = 2997705012
&A[2]: Using %p = 0x7fffb2ad5938, Using %u = 2997705016
&A[3]: Using %p = 0x7fffb2ad593c, Using %u = 2997705020
&A[4]: Using %p = 0x7fffb2ad5940, Using %u = 2997705024

0x7fffb2ad5930 = **140736191093040** in decimal (**NOT 2997705008**)
so print with %u prints a wrong value (4 bytes of unsigned int cannot hold 8 bytes for the pointer value)

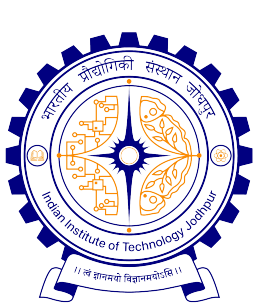
Example



double *p

Printf("%lf",*p);

Printf ("%lf",*(p-1));

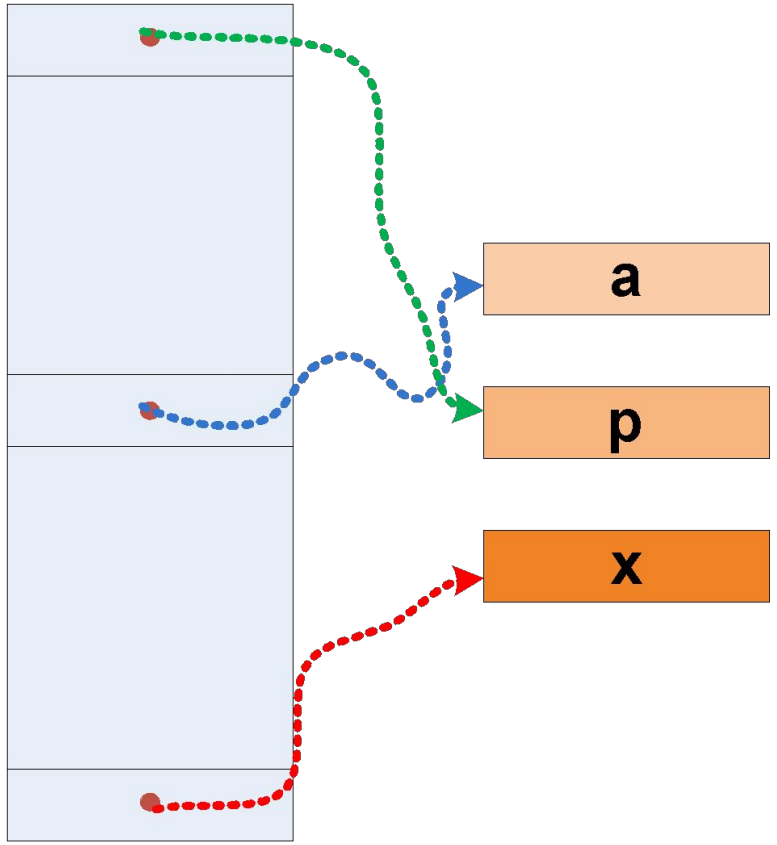


Guess the Outcome

```
int x[5] = {10, 20, 30, 40, 50};
int *p;
p = &x[1];
printf ("%d", *p);
p++;
printf ("%d", *p);
p = p + 2;
printf ("%d", *p);
```

Pointer Array

If an array stores pointers of some variables, then it is called a pointer array.




```
char * names[100];
```
// It stores pointers to 100 strings

```
int *marks[ ];
```
// It stores pointers to undefined `int` *values*

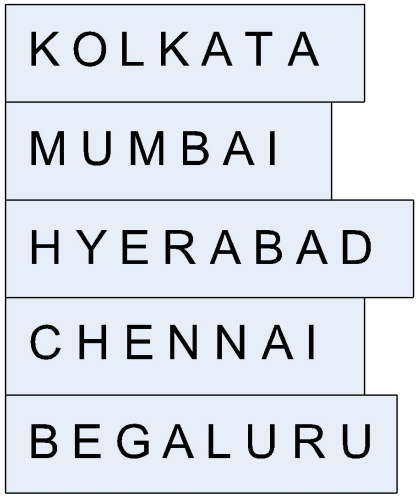
```
float **age;
```
// It stores pointers to undefined `float` *values*

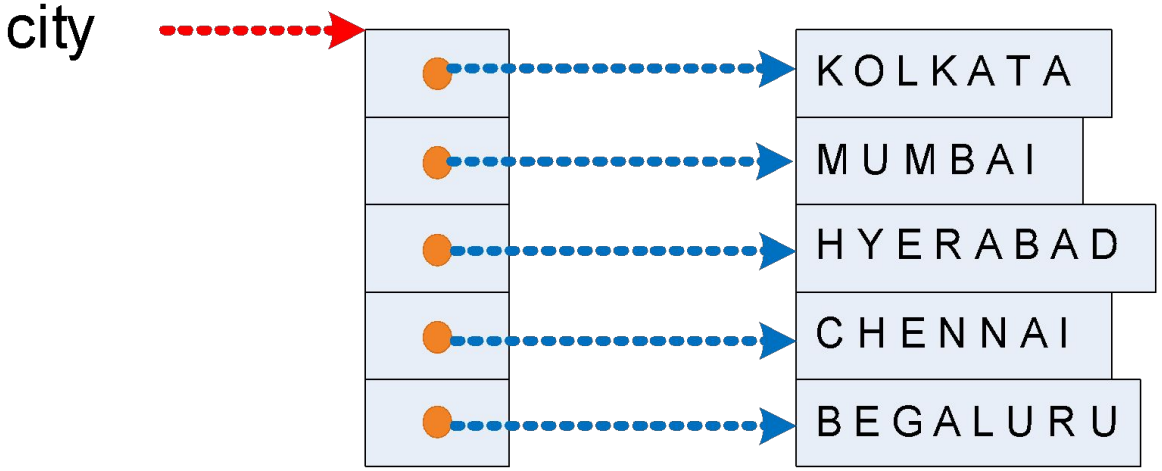
Pointer and 2D Arrays

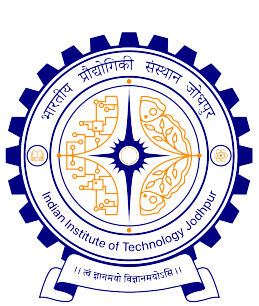
- We often use list of strings in many occasions
  - Names of cities, names of students, etc.

- Such a list of strings looks like a 2-D character array and better can be processed from the pointer view

2-D array view



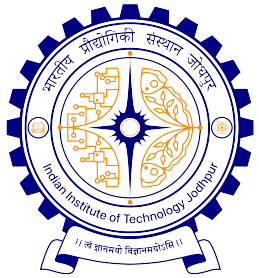




Pointer and 2D Arrays

```
char city[20][15];// 2-D array view: to store names of 20 cities of max 15 characters

char *city[20];// Pointer view: to store names of 20 cities
```

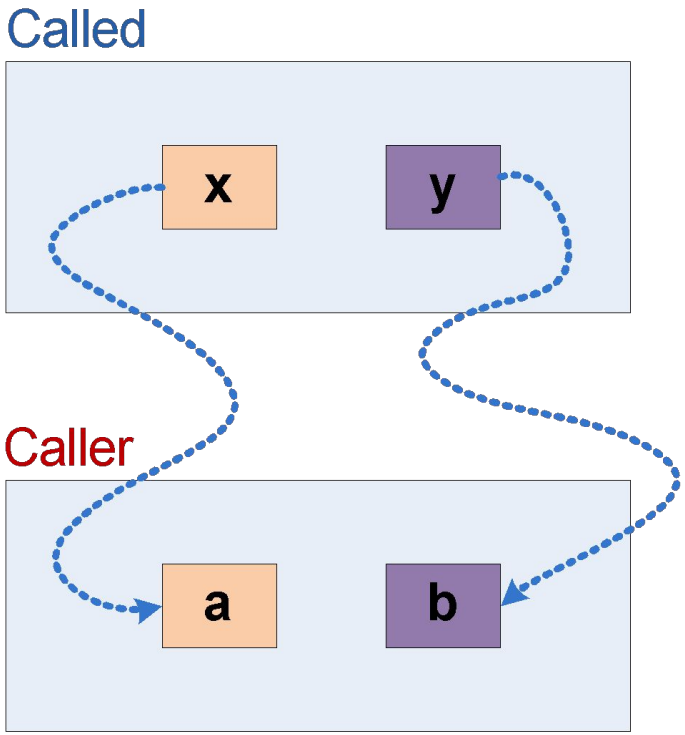
```
char *city[20] = {"Kolkata", "Mumbai", "Hyderabad",
"Chennai, Bengaluru"};        // Initialization of  pointer array

printf("%s", city[i]);        // Using 2-D array view

printf("%s", *(city+i));      // Using pointer view:w
```

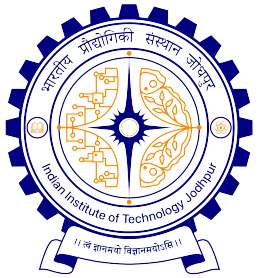


Pointers and Functions

Pointers as Function Arguments

```
#include <stdio.h>
void swap (int *x, int *y){
      *x = *x + *y;
      *y = *x – *y;
      *x = *x – *y;

      return;
}

void main(){
      int a, b;

      scanf (“a = %d, b = %d”, &a, &b);
      swap(&a, &b);

      printf (“a = %d, b = %d”, a, b);
}
```



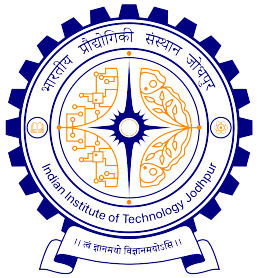


Passing array to a function

- When an array name is passed to a function, what is passed is the location of initial element.

- Within the called function, this argument is a local variable, and so an array name parameter is a pointer.

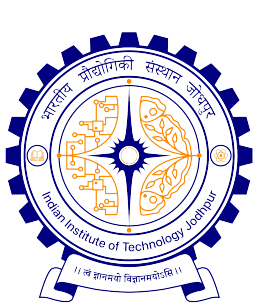
```
#include  <stdio.h>

int  arrayLen (char *s){
     int length = 0;
     for (i=0; *s != '\0'; s++)
     length++;
     return (length);
}
```



Passing array to a function

- In the example, since s is a pointer, incrementing it is perfectly legal.
- s++ has no effect on the content in the array, but just increments the private copy of the pointer.
- Following are all valid calls

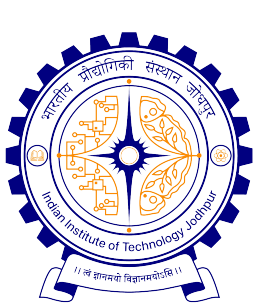
```
#include  <stdio.h>

void  main(){
     …..
     arrayLen(a);                          // When, there exist say, char a[20];
     arrayLen("I LOVE IITJ");        // Passed as a string
     arrayLen(ptr);                        // When, there exist say, char *ptr;
}
```



Example: Passing array to a function

```
int main(){
  int x[100], k, n;

  scanf ("%d", &n);

  for (k=0; k<n; k++)
     scanf ("%d", &x[k]);

  printf  ("\nAverage is %f",
                avg (x, n));
  return 0;
}
```

```
float avg (int array[], int size){
  int  *p, i , sum = 0;

  p = array;

  for (i=0; i<size; i++)
      sum = sum + *(p+i);

  return ((float) sum / size);
}
```



The pointer p can be subscripted also just like an array!

```
int main(){
  int x[100], k, n;

  scanf ("%d", &n);

  for (k=0; k<n; k++)
     scanf ("%d", &x[k]);

  printf  ("\nAverage is %f",
                  avg (x, n));
  return 0;
}
```

```
float avg (int array[], int size){
  int  *p, i , sum = 0;

  p = array;

  for (i=0; i<size; i++)
      sum = sum + p[i];

  return ((float) sum / size);
}
```

Parameter passing
Difference for array (1D, 2D)

```
int main()
{
int x;
```

x

```
…..
…….
……..
foo(x);
…

}
```

```
void foo(int a)
{
…….
……..

}
```

a

```
int main()
{
int x[20];


…..
…….
……..
foo(x);
…

}
```

```
void foo(int a[])
{
…….
……..

}
```

x

a

Parameter passing
Difference for array (1D, 2D)

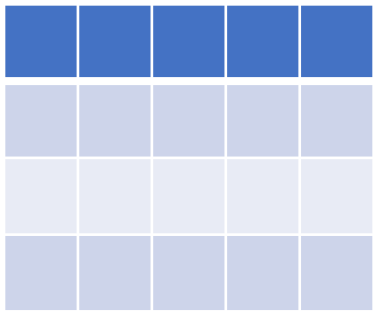
```
int main()
{
int x;
x
…..
……
……..
foo(x);
…

}
```

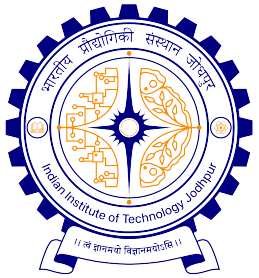
```
void foo(int a)
{
…….
……..
a

}
```

```
int main()
{
int x[20][20];
…..
……
……..
foo(x);
…

}
```

```
void foo(int a[20][20])
{
…….
……..

}
```

x
a





Pointers or Arrays in function prototype?

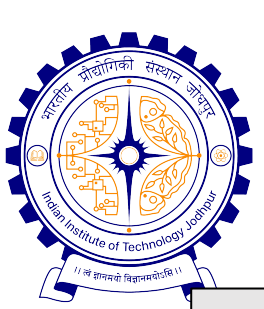
- There is no difference among the following functions prototypes.

```
... func_name ( int A[], ... );
... func_name ( int A[100], ... );
... func_name ( int *A, ... );
```

- In all the cases, A is an int pointer. It does not matter whether the actual parameter is the name of an int array or of an int pointer. Inside the function, A is a copy of the address passed.

- For readability, use the following convention.
  - If the parameter passed is a pointer to an individual item (like x = &a in the swap example), use the pointer notation in the function prototype.

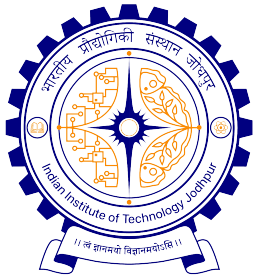
  - If the parameter passed is an array, you can use any of the two notations in the function prototype. The array notation may be preferred for readability.



A function can return a pointer

```
#include <stdio.h>
    char *firstupper ( char S[] ) {
    // You can use char *S as the formal parameter
    while (*S) if ((*S >= 'A') && (*S <= 'Z')) return S;
    else ++S;
    return NULL;
}

int main (){
char *p, S[100];
scanf("%s", S);
p = firstupper(S);
if (p)
    printf("%c found\n", *p);
else
    printf("No upper-case letter found\n");
return 0;
}
```

Note: A function should not return a pointer to a local variable. After the function returns, the local variable no longer exists.



Passing part of an array to a function

It is possible to pass a part of an array to a function.

Example:

Suppose, a function is defined as

foo(int x[]); or foo(int *x);

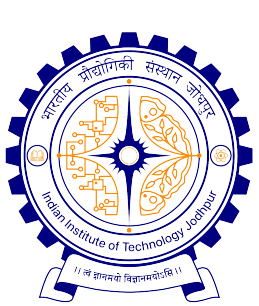
Then

foo(&a[i]);

and

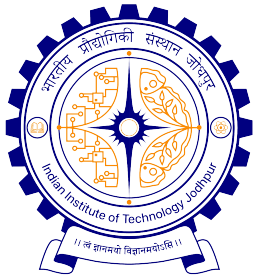
foo(a+i);

Both pass to the function foo the address of the subarray that starts at a[i].



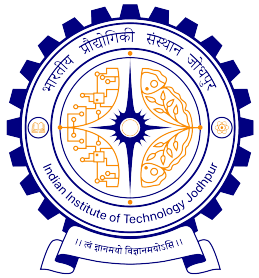
Returning ‘Multiple’ values using pointers

```
#include <stdio.h>
int f ( int a, int b, int *p, int *q ){
*p = a + b;
*q = a - b;
return a * b;
}

int main (){
    int u = 55, v = 34, x, y, z;
    z = f (u, v, &x, &y);
    printf("x = %d, y = %d, z = %d\n", x, y, z);
return 0;
}
```

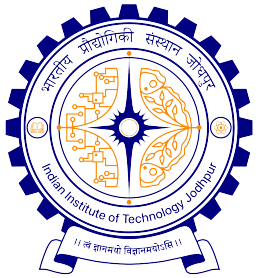


Important to remember

- Pitfall: An array in C does not know its own length & bounds not checked!
  - Consequence: While traversing the elements of an array (either using [ ] or pointer arithmetic), we can accidentally access off the end of an array (access more elements than what is there in the array)
  - Consequence: We must pass the array and its size to a function which is going to traverse it, or there should be some way of knowing the end based on the values (Ex., a –ve value ending a string of +ve values)
- Accessing arrays out of bound can cause segmentation faults
  - Hard to debug (already discussed earlier)
  - Always be careful when traversing arrays in programs



Pointers to Functions



Pointers to Functions

- **A function, like a variable, has an address location in the memory.**
- It is therefore, possible to declare a pointer to a function
  - **Which then can be used as an argument in another function**
- A pointer to a function is declared as follows

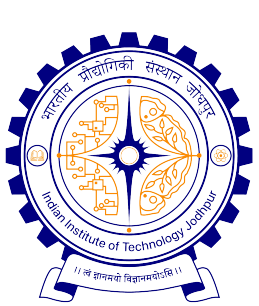
  <type> (***fptr**)(); *//This is the simplest way of declaration*

  - This tells the complier that fptr is a pointer to a function which returns <type> value
- Alternatively, a pointer to a function also can be declared as

  <type> (*fptr)(<type1>, <type2>, . . .);

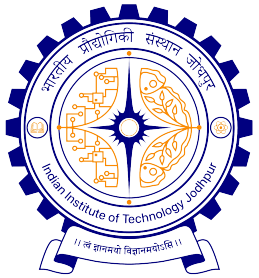
Or

  <type> (*fptr)(<type1><agrg1>, <type2><arg2>, . . .);



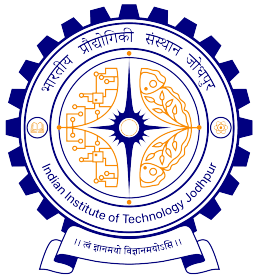
Pointers to Functions: Example

```
double (*p)();      // p is a pointer to a function
double add(double x, double y);  // a function declaration
 ......
 p = add;           // pointer to function is assigned to p
      ... ...
 add(a,b);          // The function add() is called with its arguments

(*p)(a,b);          // This is equivalent to  add(a,b)
```



Passing Functions to other Functions

- A pointer to a function (say, *guest*) can be passed to another function (say, *host*) as an argument.
  - This allows the guest function to be transferred to host, as though the guest function were a variable.
- Successive calls to the host function can pass different pointers (i.e., different guest functions) to the host.
- When a host function accepts the name of a guest function as an argument, the formal argument declaration must identify that argument as a pointer to the guest function.



Passing Functions to other Functions

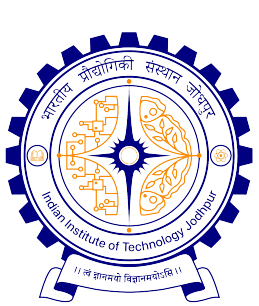
- **Declaring a guest function**
  - Guest function can be declared as usual

```
<type_g> gfName(<type1>(arg1), <type2><arg2>, . . .);
```

- **Declaring a host function**
  - Host function can be declared as usual

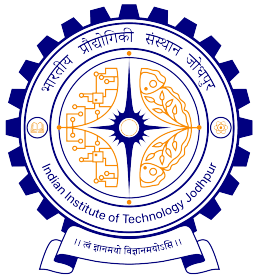
```
<type_h> hfName(..., <type_g>(*)( ),
        . . . data types of other arguments in host);
```

- Note that the indirection operator '*' appears in parenthesis, to indicate a pointer to the guest function. Moreover, the data types of the guest function's arguments follow in a separate pair of parenthesis, to indicate that they are the function's arguments.

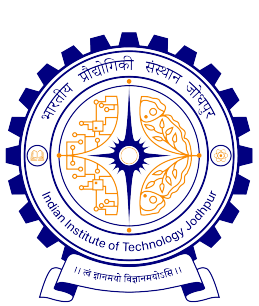


Passing Functions to other Functions

```
#include<stdio.h>
int process(int, int(*)(), char *);  // Declaration of a host function
int func1(int, int);                 // Declaration of a guest function
int func2(int, int);                 // Declaration of another guest function

void main(){
   int i, j; char name[20]);
   ...
    i = process(i, func1, name);
   ...
    j = process(j, func2, name);

    return;
}
```

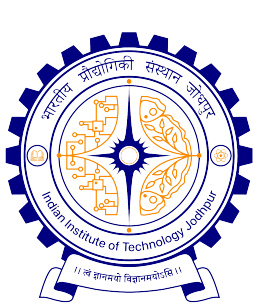



```
int process(int x, int (*pf)(), char *s) {
        int a, b, c;
        ...
        c = (*pf)(a, b);                    // Access the guest function
   ...
   return(c);
}
int func1(int a, int b) {
      int c;
      ...                                   // Code in the guest function
      return(c);
}
int func2(int a, int b) {
       int d;
      ...                                   // Another code in the guest
function
      return(d);
}
```



Exploring void* with scanf()

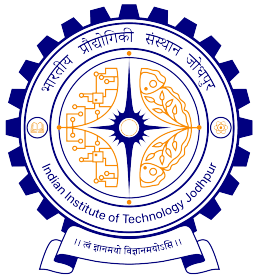
- Prototype: *int scanf(char* str, void*, void*, ...);*
- What is void*?
  - it can point at anything
- Since the data types being passed into scanf can be anything, we need to use void* pointers
- If you want to scan a value into a local variable, you need to pass the address of that variable
  - this is the reason for the ampersand (&) in front of the variable



void * as argument to a function

```
void printValue(void *data, char type) {
    if (type == 'i') {
        printf("Integer value: %d\n", *(int*)data); // Cast to int* and dereference
    } else if (type == 'f') {
        printf("Float value: %.2f\n", *(float*)data); // Cast to float* and dereference
    } else if (type == 'c') {
        printf("Character value: %c\n", *(char*)data); // Cast to char* and dereference
    } else {
        printf("Invalid data type\n");
    }
}
```

```
int main() {
    int intValue = 42;
    float floatValue = 3.14;
    char charValue = 'A';

    // Pass the address and a type indicator to the generic function

    printValue(&intValue, 'i');
    printValue(&floatValue, 'f');
    printValue(&charValue, 'c');
return 0;
}
```



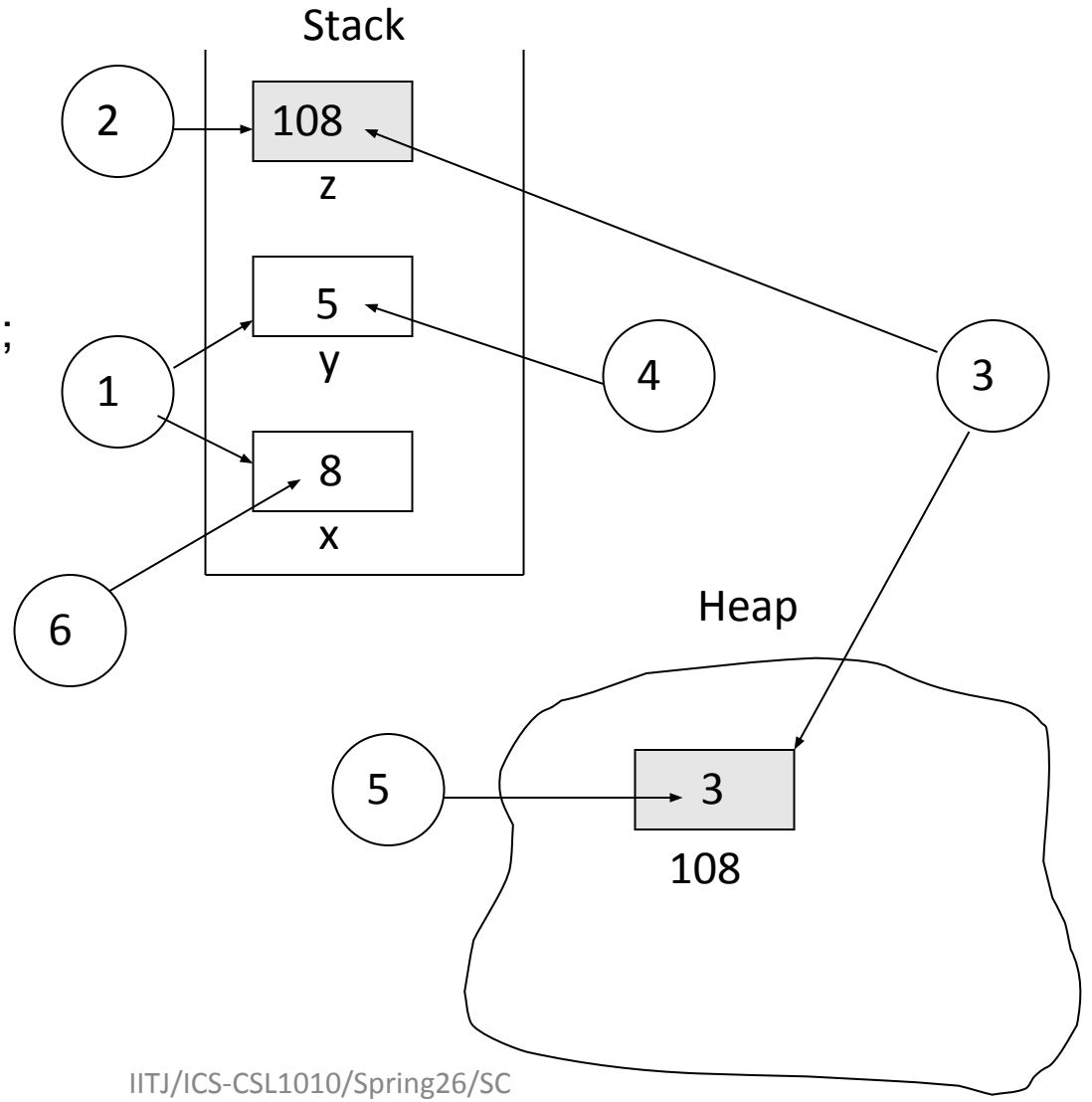
Memory Allocation for Pointer Variable

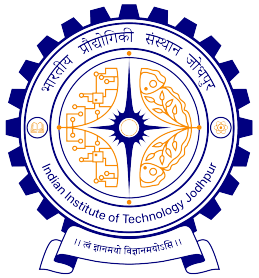
- Ideally, a pointer is simply a local variable that refers to a memory location on the heap
- Accessing the pointer, actually references the memory on the heap
- The heap is an area of memory that the user handles explicitly
  - user requests and releases the memory through system calls
  - if a user forgets to release memory, it doesn't get destroyed
    - it just uses up extra memory
- A user maintains a handle on memory allocated in the heap with a *pointer*
- *D*eclaring a variable does not allocate space on the heap for it
    - it simply creates a local variable (on the stack) that will be a pointer
    - use *malloc()* to actually request memory on the heap

Example

Next, we will discuss dynamic memory allocation

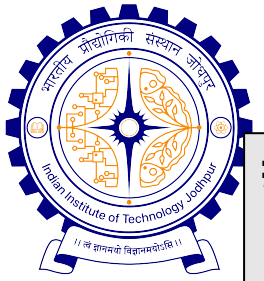
```
   int main() {
1      int x, y;
2      int *z;
3      z = malloc(sizeof(int));

4      y = 5;
5      *z = 3;
6      x = *z + y;
7      free(z);

       return 0;
   }
```





Guess the Outcome

```
#include <stdio.h>
int a[] = {0, 1, 2, 3, 4};
int main(){
        int i, *p;

        for (i=0; i<=4; i++)
              printf("i=%d:, a[i]=%d\t",i, a[i]);
              printf("\n");

        for (p=&a[0]; p<=&a[4]; p++)
              printf("*p = %d\t", *p);
              printf("\n");

        for (p=&a[0], i=1; i<=5; i++)
              printf("i=%d:, p[i] = %d, *p= %d\t",i, p[i], *p);
              printf("\n");
 return 0;
}
```



Guess the Outcome

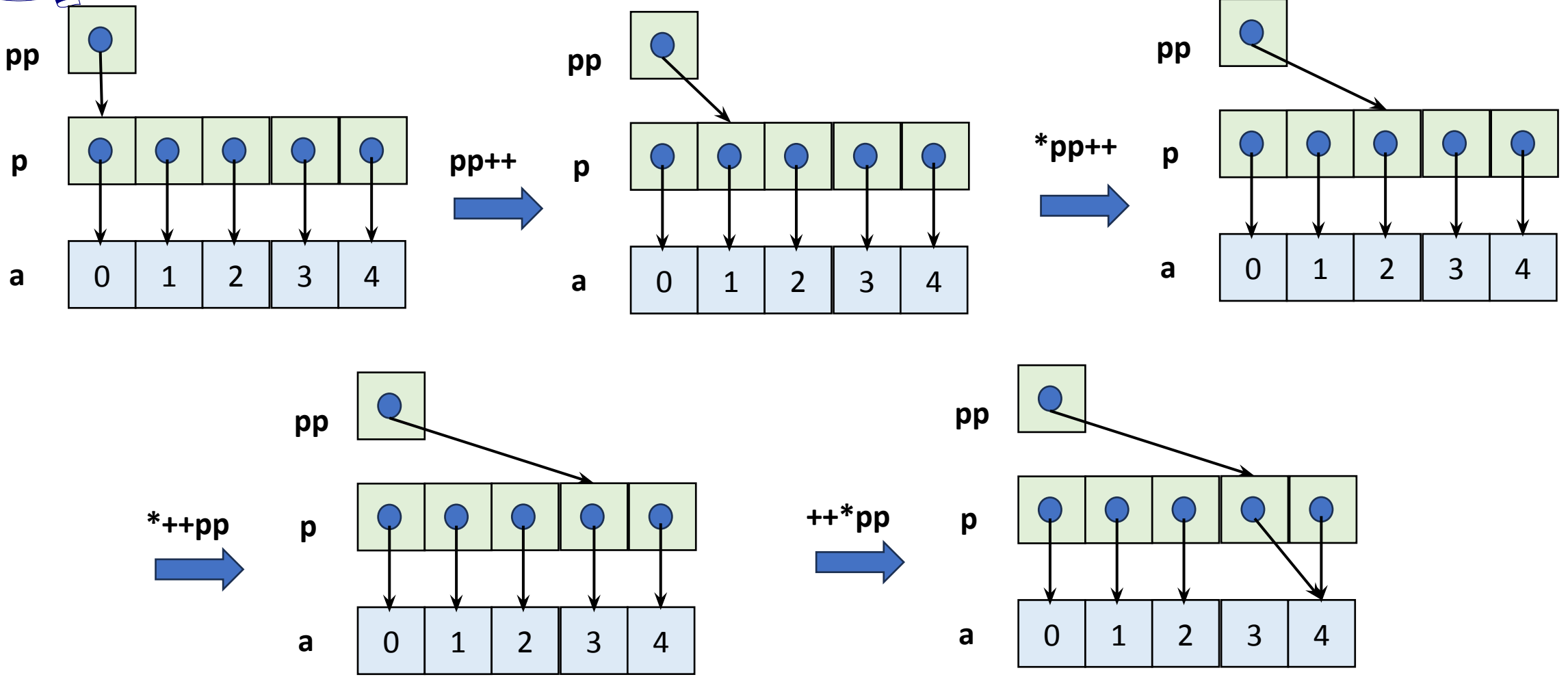
```
#include <stdio.h>
int a[] = {0, 1, 2, 3, 4};
int main(){
        int i, *p;
        for (p=a, i=0; p+i<=a+4; p++, i++)
                printf("i=%d:, *p+i = %d, *(p+i)= %d\t",i, *p+i, *(p+i));
                printf("\n");

        for (p=a+4; p>=a; p--)
                printf("*p= %d\t",*p);
                printf("\n");

        for (p=a+4, i=0; i<=4; i++)
                printf("p[-i] = %d\t", p[-i]);
                        printf("\n");

        for (p=a+4; p>=a; p--)
                printf("p=%p:,a=%p:;p-a=%lu:,a[p-a]= %d\t",p,a,p-a,a[p-a]);
                printf("\n");
return 0;
}
```

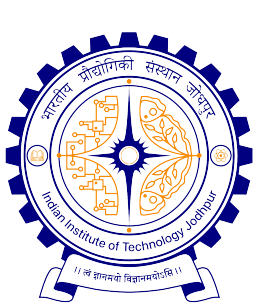
Guess the Outcome

```
#include <stdio.h>
int a[] = {0, 1, 2, 3, 4};
int *p[] = {a, a+1, a+2, a+3, a+4};
int **pp = p;

int main(){
        printf("a=%lu:, *a=%lu\n",a,*a);
        printf("p=%lu:, *p=%lu:, **p=%lu\n",p, *p, **p);
        printf("pp=%lu:, *pp=%lu:, **pp=%lu\n",pp, *pp, **pp);

return 0;
}
```

Guess the Outcome

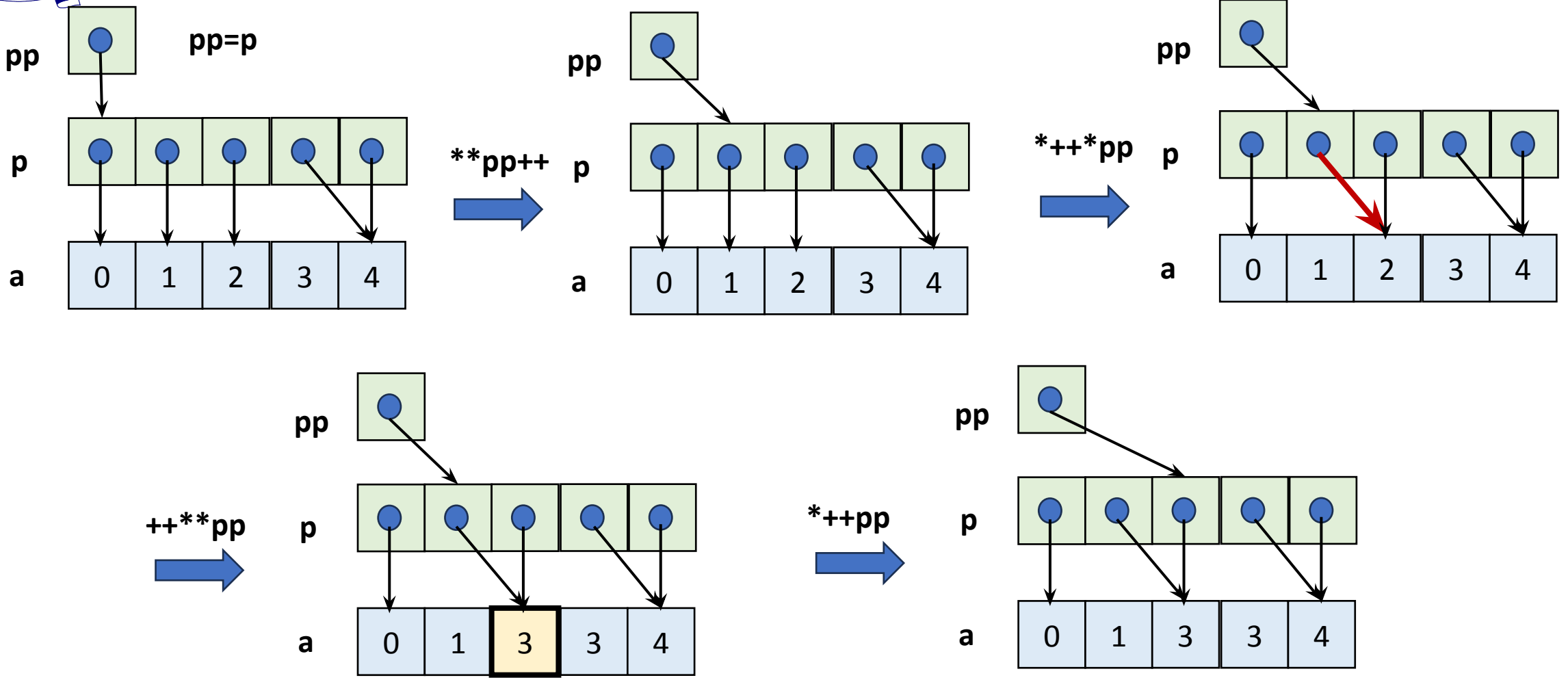
```
#include <stdio.h>
int a[] = {0, 1, 2, 3, 4};
int *p[] = {a, a+1, a+2, a+3, a+4};
int **pp = p;

int main(){
   pp++;
   printf("pp-p=%d:, *pp-a=%d:, **pp=%d\n", pp-p, *pp-a, **pp);
   *pp++;
   printf("pp-p=%d:, *pp-a=%d:, **pp=%d\n", pp-p, *pp-a, **pp);
   *++pp;
   printf("pp-p=%d:, *pp-a=%d:, **pp=%d\n", pp-p, *pp-a, **pp);
   ++*pp;
   printf("pp-p=%d:, *pp-a=%d:, **pp=%d\n", pp-p, *pp-a, **pp);
```

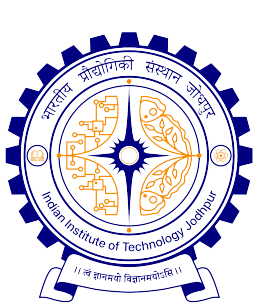

pp
p
a
0
1
2
3
4
pp++
pp
p
a
0
1
2
3
4
*pp++
pp
p
a
0
1
2
3
4
*++pp
pp
p
a
0
1
2
3
4
++*pp
pp
p
a
0
1
2
3
4



Guess the Outcome

```
   pp=p;
   **pp++; printf("pp-p=%d:, *pp-a=%d:, **pp=%d\n", pp-p,
*pp-a, **pp);
   *++*pp; printf("pp-p=%d:, *pp-a=%d:, **pp=%d\n", pp-p,
*pp-a, **pp);
   ++**pp; printf("pp-p=%d:, *pp-a=%d:, **pp=%d\n", pp-p,
*pp-a, **pp);
   *++pp; printf("pp-p=%d:, *pp-a=%d:, **pp=%d\n", pp-p, *pp-a,
**pp);
return 0;
}
```


pp
pp=p
p
a
0
1
2
3
4
**pp++
pp
p
a
0
1
2
3
4
*++*pp
pp
p
a
0
1
2
3
4
++**pp
pp
p
a
0
1
3
3
4
*++pp
pp
p
a
0
1
3
3
4



Practice Problem Set

1. Is the following correct? If not, why?

   float x; int *p; p = &x;

2. What amount of memory would be require to store the values of the two pointer variables p and q as declared below?

   float x, *p; int y, *q;

3. Is the following statements valid? If not, why?

   int *count; count = 1024;

4. How one can print the a) value stored in a pointer variable and b) memory location, where a pointer variable is stored? What about c) the memory location of the pointer variable itself?

5. A pointer essentially stores a memory location. Now, amount of byte to store a memory location of a variable of any type is same. Does a pointer array can store pointers of any type of variables?

Practice Problem Set…

6. Consider the following declarations and answer the following.

int *x, *p; char a[20]; p = &x; x = 100;

a) How to print the address of x:

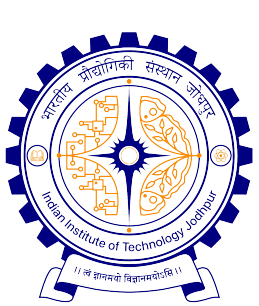
b) What *p indicates?

c) How to print the address where p is stored?

d) How to print the content (i.e., address) which is stored in p?

e) How to access the variable through p?

f) can we write p = 0xabc6, where the right-hand side is a hexa-decimal number?

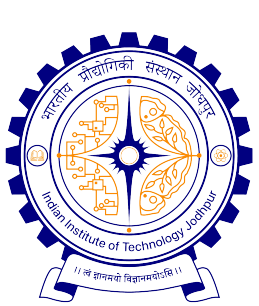


Practice Problem Set…

7. Given that **int *p, x, y;**

Tell (in terms of equivalent C statement) the implication of the following.

```
a)  p = &x;          y = *p:

b)  y = *&x;

c) *p = 255;

d) p++;

e) y += *p;
```



Practice Problem Set…

8. Consider the following and state which of the given statements are illegal

```
int a[10];
int *p;
```

a) `p = a;`

b) `a = p;`

c) `a + i;`

d) `a ++;`

e) `p + i;`

Practice Problem Set…

9. Consider the following program. What the output a, b, c, …, i signify? Draw an appropriate memory instance and the explain your answers.

```
include <stdio.h>
int main() {
  int i = 3, *j, **k;
  j = &i; k = &j;
  printf("a is %u", &i);
  printf("b is %u", j);
  printf("c is %u", k);
  printf("d is %u", &j);
  printf("e is %u", &k);
  printf("f is %d", i);
  printf("g is %d", *j);
  printf("h is %d", *(&i));
  printf("i is %d", **k);
return 0;
}
```